A3: Reversing the AI-Slop Feedback Loop with Barrier-Certificate-Guided Static Analysis for Python

Anonymous Author(s)

Abstract

Large language models (LLMs) can generate candidate bug-detection heuristics, verification hypotheses, and even static analysis rules at unprecedented speed—but the vast majority of such AI-generated material is imprecise, unsound, or outright wrong. Unchecked, this creates a *negative* feedback loop—the *AI-slop feedback loop*—in which an LLM proposes analysis ideas, most are noise, that noise pollutes the next round of prompts and training data, and quality degrades with every iteration.

We present A3 (**A**dvanced **A**utomated **A**nalysis), an open-source Python static-analysis tool that *inverts* this loop, turning it from a vicious cycle into a virtuous one. A3 combines (1) bytecode-level control-flow and data-flow analysis, (2) a portfolio of 18 verification techniques drawn from the barrier-certificate, CEGAR, IC3/PDR, SOS/SDP, CHC, and invariant-synthesis literatures (the *kitchensink* engine), and (3) agentic LLM triage that investigates only the small residual set of findings that static proof and disproof both leave unresolved.

On a synthetic benchmark suite of 43 injected bugs across 67 bug types, A3 achieves 90.2% precision at 100% recall ($F_1 = 0.949$). On Microsoft DeepSpeed (~150 k LoC, 7 826 functions), the barrier-certificate engine alone proves 95.3% of 2 638 raw findings to be false positives, leaving 10 confirmed true bugs including 6 division-by-zero vulnerabilities reachable from user-supplied configuration. We further evaluate A3 against the BugsInPy benchmark of 493 real bugs across 17 open-source projects and describe an iterative self-improvement protocol in which false negatives and false positives drive automated patches to the analyser itself. Beyond BugsInPy, A3 generalises this protocol to *arbitrary* Python repositories: given any repo's git history, it automatically identifies bug-fix commits, compares the buggy and fixed versions, and uses misclassifications to drive further analyser improvements—enabling continuous verification that strengthens the analyser with every new commit it encounters. Across 15 Microsoft-maintained repositories, A3 surfaces true-positive security and crash bugs in Counterfit, RD-Agent, RESTler, FLAML, and Presidio, among others.

ACM Reference Format:
Anonymous Author(s). 2026. A3: Reversing the AI-Slop Feedback Loop with Barrier-Certificate-Guided Static Analysis for Python. In *Proceedings of 41st IEEE/ACM International Conference on Automated Software Engineering (ASE '26).* ACM, New York, NY, USA, 9 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
ASE '26, Sacramento, CA, USA
© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 Introduction

The last three years have produced a remarkable acceleration in AI-assisted software engineering. LLMs write code, generate tests, propose refactorings, and even suggest formal specifications. Yet a quieter, less-celebrated failure mode has emerged alongside these capabilities: *AI slop*—the unchecked proliferation of plausible-sounding but incorrect or imprecise artifacts generated by language models.

When an LLM is asked to produce static-analysis rules, verification hypotheses, or bug-detection heuristics, it generates *broadly and quickly*. The results are rarely sound. In our experience, a single prompting session can yield dozens of candidate bug-detection rules of which fewer than 20% survive contact with a real codebase. The remaining 80% produce false positives, miss edge cases, or encode assumptions that are semantically vacuous once bytecode-level control flow is considered.

Left unchecked, this process compounds into what we call the *AI-slop feedback loop*—a **vicious cycle**:

(1) AI-generated rules flood the analyser with noise.
(2) Developers lose trust and disable alerting.
(3) The feedback signal for improving the LLM's future suggestions degrades.
(4) The next generation of AI-generated rules is trained on, or prompted with, the output of the noisy system, producing *even more* noise.

This paper describes A3, a system that *flips this loop on its head*: by inserting rigorous, deterministic static-analysis machinery between the LLM's expansive generation and the developer's inbox, the same iterative cycle becomes a **virtuous cycle** in which each round of AI-generated hypotheses is pruned by formal methods, the survivors improve the analyser, and the improved analyser produces sharper feedback for the next round. The core architectural principle is **static-first, agentic-second**: deterministic, auditable, non-LLM reasoning handles the vast majority of work, and LLM judgment is reserved for the small residual where static proof and disproof both stop.

A3 is implemented as an open-source Python package (`pip install a3-python`), totalling roughly 191 k lines of Python across 67 bug-type detectors, a multi-technique verification portfolio, and a CI integration layer with baseline ratcheting.

Contributions.

- We characterise the AI-slop feedback loop—historically a degenerative vicious cycle—and show how a five-phase iterative methodology (theorise → code → test → fix code → fix theory) inverts it into a virtuous self-improvement cycle (§2).
- We present A3's *kitchensink* verification engine, which orchestrates 18 SOTA techniques from the barrier-certificate, CEGAR, IC3/PDR, SOS/SDP, CHC, ICE, Houdini, SyGuS,

and assume-guarantee literatures into a unified false-positive filter (§3).
- We describe a self-improvement protocol that uses the BugsInPy benchmark [1] of 493 real bugs across 17 open-source Python projects to iteratively patch the analyser with the help of an LLM coding assistant (§4).
- We report quantitative results on synthetic benchmarks ($F_1 = 0.949$), large-scale industrial code (DeepSpeed: 95.3% FP elimination, 10 confirmed TPs), and a portfolio of 15 Microsoft-maintained repositories with confirmed true positives (§6).
- We show how the self-improvement protocol generalises beyond curated benchmarks to arbitrary Python repositories by mining bug-fix commits from git history, enabling *continuous verification* in which the analyser improves with every repository it scans (§5).

2 The AI-Slop Feedback Loop: From Vicious to Virtuous

2.1 Origin: From Quantitative Semantics to Shipping Code

The project that became A3 did not begin as a CLI tool. It began with an almost unreasonable challenge: could an LLM generate a genuinely novel verification theory at research-paper scale, then survive contact with implementation?

The earliest phase produced a 74-page manuscript on quantitative model checking. The central idea was to stop asking only "is the system safe?" and instead ask "how far is it from safety?"—treating the distance to a safety boundary as a first-class semantic object that can be computed, optimised, and used to prioritise analysis effort. Three instincts from this phase survived into the final system:

(1) Safety is not merely Boolean; uncertainty has geometric *shape*.
(2) Quantitative semantics can *prioritise* analysis work.
(3) If distance to a safety boundary is computable, *repair* can be guided.

What did not survive was the assumption that quantitative semantics alone suffices for industrial-scale bug detection. In practice, most engineering pain is not proving one deep theorem but *killing mountains of false positives without missing true bugs.*

2.2 Three Eras

The project passed through three eras, each surviving as a layer in the final architecture:

Era I: Distance to Safety. Traces modelled as distributions, properties as structured acceptance sets, distance to satisfaction as a metric. This era contributed the *confidence scoring* module.

Era II: Barrier Certificates. The system shifted from measurement to separation: construct a mathematical function $B(\mathbf{x})$ such that initial states satisfy $B(\mathbf{x}) \leq 0$, unsafe states satisfy $B(\mathbf{x}) > 0$, and every transition preserves the sign invariant. If such a B exists, safe and unsafe regions are provably disjoint. This era contributed the core false-positive elimination engine.

Era III: Python Reality. Making the theory survive Python's execution semantics—bytecode-level control flow, normal and exceptional edges, frame/stack state, dynamic dispatch, unknown library behaviour—required committing to implementation details that killed many elegant claims. Good: they needed to die.

2.3 Inverting the Loop: The Five-Phase Iteration

The AI-slop feedback loop, left to its own devices, is degenerative: each iteration adds noise. The key insight behind A3 is that the same generative breadth that causes the vicious cycle can power a *virtuous* one—provided every AI-generated hypothesis is forced through a gauntlet of formal and empirical checks before it reaches production. What emerged was a five-phase iteration that was not planned but was repeated often enough to become the project's real contribution:

(1) **AI Theorising.** An LLM generates broad hypotheses: new abstractions, candidate proof templates, cross-domain analogies, aggressive architectural combinations.
(2) **Coding.** Ideas are encoded in analysers: bytecode/CFG extraction, symbolic state propagation, unsafe-region checks, barrier template synthesis, dynamic symbolic/concolic validation.
(3) **Testing.** The expensive truth step: synthetic suites, regression tests, large-repo scans, confidence calibration, triage audits.
(4) **Fixing Code.** Typical breakages: path explosion, over-conservative unknown-call handling, context loss across call boundaries, duplicate floods, false positives on guard-heavy code.
(5) **Fixing Theory.** The underappreciated step: instead of forcing code to match a brittle theory, the theory itself is patched—definitions tightened, assumptions made explicit, proof obligations split by semantics layer.

The loop then restarts—but unlike the unchecked slop loop, each iteration *improves* the system rather than degrading it. This is the inverted loop in practice: generate aggressively, then subject everything to adversarial execution. The asymmetry is what makes the cycle virtuous rather than vicious: upstream AI *expands* the hypothesis space; midstream formal/static machinery *prunes* it brutally; downstream agentic AI handles the hard residue. Because the pruning is deterministic and auditable, noise is destroyed rather than propagated, and the surviving signal feeds back into better hypotheses in the next round.

3 The A3 Analysis Engine

3.1 Architecture Overview

A3's pipeline has seven stages:

(1) **Call-graph construction** from Python bytecode.
(2) **Crash-summary computation**: per-function summaries of operations that can raise (division, attribute access, indexing, assertions).
(3) **Guard detection**: identification of explicit checks (`if x is not None`, `isinstance`, `try/except`) that render a potential crash unreachable.

(4) **Barrier-certificate proofs**: 8 barrier patterns that compositionally prove false positives (assume-guarantee contracts, refinement types, inductive invariants, factory post-conditions, disjunctive barriers, control-flow barriers, dataflow barriers, callee return-guarantee safety).
(5) **Kitchensink portfolio verification**: orchestration of 18 SOTA techniques for the surviving candidates.
(6) **Z3-backed DSE confirmation**: directed symbolic execution to confirm or refute surviving findings.
(7) **Deduplication and scoring**: confidence-weighted ranking and SARIF output.

Bytecode-level analysis. Unlike tools that operate on the AST, A3 analyses compiled Python bytecode (`.pyc` files or in-memory `code` objects obtained via `compile()`). This decision has three consequences. First, bytecode is the ground truth that CPython actually executes, so the analyser's model of control flow matches the interpreter's. Second, bytecode makes exception edges explicit: every `BINARY_OP`, `LOAD_ATTR`, and `CALL` instruction can transfer to an exception handler, and those edges are visible in the bytecode's exception table (Python 3.11+). Third, bytecode normalises syntactic sugar—`with` statements become `SETUP_WITH`/`BEFORE_WITH` instructions, comprehensions become nested code objects—eliminating a class of AST-level ambiguities.

The control-flow graph (CFG) is constructed by linear scanning of bytecode instructions, splitting at branch targets, exception boundaries, and `RETURN_VALUE`/`RAISE_VARARGS` instructions. Call-graph edges are resolved by tracking `LOAD_GLOBAL`, `LOAD_ATTR`, and `LOAD_DEREF` instructions that resolve to known callables, with conservative fallback for dynamic dispatch.

Interprocedural analysis. For crash and exception bugs, A3 computes per-function *crash summaries*: a compact representation of the conditions under which a function can raise an unhandled exception. Summaries are propagated bottom-up through the call graph: if function `f` calls `g`, and `g`'s summary indicates that it can raise `ZeroDivisionError` when its second argument is zero, then `f`'s summary inherits that condition, weakened by any guards `f` places on the argument before the call. This summary-based approach avoids full-program inline expansion while preserving precision for the most common interprocedural patterns (argument forwarding, return-value propagation, exception re-raising).

3.2 Bug Taxonomy

A3 implements detectors for 67 bug types organised in four families:

- **Crash bugs (19 types):** `NULL_PTR`, `DIV_ZERO`, `BOUNDS`, `TYPE_CONFUSION`, `ASSERT_FAIL`, `STACK_OVERFLOW`, `USE_AFTER_FREE`, `UNINIT_MEMORY`, `DOUBLE_FREE`, `INTEGER_OVERFLOW`, `FP_DOMAIN`, `MEMORY_LEAK`, `NON_TERMINATION`, `ITERATOR_INVALID`, `DATA_RACE`, `DEADLOCK`, `SEND_SYNC`, `INFO_LEAK`, `TIMING_CHANNEL`.
- **Security bugs (39 types):** Injection (`SQL_INJECTION`, `COMMAND_INJECTION`, `CODE_INJECTION`, `PATH_INJECTION`, ...), web (`REFLECTED_XSS`, `SSRF`, `CSRF_PROTECTION_DISABLED`), serialisation (`UNSAFE_DESERIALIZATION`, `XXE`), cryptography (`WEAK_CRYPTO_KEY`, `BROKEN_CRYPTO_ALGORITHM`), filesystem and network.
- **Exception bugs (fine-grained):** `VALUE_ERROR`, `RUNTIME_ERROR`, `FILE_NOT_FOUND`, `PERMISSION_ERROR`, `OS_ERROR`, `IO_ERROR`, `IMPORT_ERROR`, etc., each mapped to optimal verification strategies from the kitchensink portfolio.
- **Semantic bugs (24 types):** Contract violations (`PRECONDITION_VIOLATIO`, `LISKOV_VIOLATION`), temporal properties (`USE_BEFORE_INIT`, `USE_AFTER_CLOSE`), data flow (`UNVALIDATED_INPUT`, `UNCHECKED_RETURN`).

3.3 The Kitchensink Portfolio

A single verification technique is rarely sufficient across all 67 bug types. A3's *kitchensink* engine (enabled by default; disable with `--no-kitchensink`) treats published verification methods as interoperable components in a verification control loop:

(1) Classify the bug's semantic shape.
(2) Route to the strongest low-cost method first.
(3) Escalate only when proof/counterexample remains unresolved.
(4) Keep competing methods as cross-checks, not decorations.

The portfolio draws from:

- Barrier-certificate foundations for safety separation [2, 3].
- Algebraic proof machinery: Positivstellensatz, SOS/SDP, hierarchy lifting, sparsity, and DSOS/SDSOS speed layers [4–8].
- Property-directed and CHC-style reachability: IC3/PDR [9], Spacer [10], interpolation-based model checking [11].
- Abstraction-refinement: classic CEGAR [12], predicate abstraction [13], lazy interpolation [14].
- Learning/synthesis: ICE [15], Houdini [16], SyGuS [17].
- Compositional reasoning: assume-guarantee [18].

3.4 Multi-Barrier Composition

The core false-positive elimination theorem is a disjunctive composition of 8 barrier patterns:

$$B_{\text{total}} = B_1 \vee B_2 \vee \cdots \vee B_8$$

If *any* barrier B_i holds for a candidate finding, that finding is proven to be a false positive. The 8 patterns are:

B_1: **Assume-guarantee contracts.** If every caller of a function satisfies a precondition that makes the bug unreachable (e.g., every call site passes a non-zero divisor), the function's division-by-zero warning is a false positive. A3 checks this by inspecting all call sites in the call graph and verifying that the precondition holds at each.

B_2: **Refinement types.** Type narrowing via `isinstance` checks, `assert` statements, and conditional branches restricts the set of values a variable can hold. If the narrowed type excludes the bug-triggering value (e.g., `isinstance(x, int)` followed by `x > 0`), the bug is unreachable.

B_3: **Inductive invariants.** Loop invariants and class invariants that are established by $_{i}nit_{a}ndpreservedbyallmethods.Forexample,ifself.n_heads is$

B_4: **Factory post-conditions.** If a value is produced by a factory function or constructor whose return type guarantees non-nullness (e.g., `dict.get(k, default)` with a non-None default), attribute access on that value cannot raise `AttributeError`.

B_5: **Disjunctive barriers.** A combination of partial barriers that individually are insufficient but together cover all paths to the bug. For example,

one branch checks `x is not None` and another checks `len(x) > 0`; neither alone proves safety, but their disjunction does.

B_6: **Control-flow barriers.** Unreachability proved by control-flow dominance: if the only path to the bug site passes through a return, raise, or `sys.exit()` that always executes first, the bug is dead code.

B_7: **Dataflow barriers.** Value-flow analysis showing that the bug-triggering value is always overwritten before reaching the bug site. For instance, a variable is initialised to `None` but is always reassigned by a preceding loop or conditional before use.

B_8: **Callee return-guarantee safety.** If a callee's return value is guaranteed non-null (because the callee always returns a concrete value or raises before returning `None`), then null-pointer dereferences on the return value are false positives. This pattern eliminated the last 27 interprocedural null-pointer FPs in the DeepSpeed evaluation.

Compositionality. The barriers are checked in order of cost: B_6 and B_7 (control-flow and dataflow) are cheapest because they require only intraprocedural analysis; B_1 and B_8 (assume-guarantee and callee return) are most expensive because they require call-graph traversal. The disjunctive composition means that once any B_i fires, the remaining barriers are skipped for that finding, keeping overall analysis time sublinear in the number of patterns.

On DeepSpeed, this composition eliminates 2 513 of 2 638 raw findings (95.3%), with 100% of interprocedural null-pointer warnings proven false positive.

4 Self-Improvement via BugsInPy

4.1 Background

BugsInPy [1] is a curated database of 493 real bugs extracted from 17 popular open-source Python projects: Ansible, Black, Cookiecutter, FastAPI, HTTPie, Keras, Luigi, Matplotlib, Pandas, PySnooper, Sanic, Scrapy, SpaCy, thefuck, Tornado, tqdm, and youtube-dl. Each bug consists of a buggy commit, a fixed commit, and a minimal test case that exposes the bug.

4.2 Scope Classification

Not every BugsInPy bug is in scope for a static analyser. A3 targets crash bugs (missing null checks, division by zero, out-of-bounds access), security vulnerabilities (injection, deserialisation, path traversal), and exception-related semantic bugs (unguarded `AttributeError`, `TypeError`, `KeyError`, `IndexError`, `ValueError`). Out-of-scope categories include pure logic/algorithmic bugs (wrong formula, wrong constant), formatting/string-output bugs, performance bugs, API-design changes, and build/packaging reorganisations.

We developed a heuristic classifier (`classify_bugsinpy_scope.py`) that analyses each bug's patch to determine whether it falls within A3's detection capabilities, using AST analysis and keyword heuristics over the diff. The classifier examines: (1) whether the patch modifies exception-handling code (`try`/`except`/`raise`); (2) whether added lines contain guard checks (`if x is None`, `isinstance`, bounds checks); (3) whether the bug's test case asserts a specific exception type; and (4) whether the patch touches security-relevant APIs (`os.system`, `pickle.load`, `eval`, SQL string formatting). Bugs matching any of these signals are classified as in-scope; the remainder are labelled out-of-scope.

Across the full BugsInPy corpus, the classifier labels approximately 40% of bugs as in-scope—predominantly from projects with rich exception-handling patterns (Ansible, Tornado, Scrapy) and fewer from projects whose bugs are primarily algorithmic (Black, tqdm).

4.3 Iterative Self-Improvement Protocol

The self-improvement loop operates as follows:

(1) **Extract.** For each in-scope BugsInPy bug, extract the changed Python file(s) from the patch.
(2) **Scan.** Run A3 on both the buggy and fixed versions of each changed file.
(3) **Classify.** Compare results against ground truth:
   - *True positive*: A3 reports a bug in the buggy version and reports clean in the fixed version.
   - *False negative*: A3 misses the bug (buggy version scans clean).
   - *False positive*: A3 flags the fixed code.
(4) **Improve.** For every false negative or false positive, construct a structured prompt describing the gap and invoke an LLM coding assistant (GitHub Copilot CLI) to patch A3's implementation.
(5) **Verify.** Re-scan the bug to confirm the patch resolves the classification error before proceeding.

This protocol is automated end-to-end (`iterative_bugsinpy_improve.py`). The outer loop repeats indefinitely (interrupted by Ctrl-C); the inner loop iterates over BugsInPy bugs, and on the first false positive or false negative, invokes the LLM to patch A3, then re-scans to verify the fix. State is checkpointed to JSON so the process can be resumed across sessions.

The key insight is that *prior git changes to the analyser*—accumulated over hundreds of iterations of the five-phase loop—serve as training signal for future improvements. Each patch to A3 narrows the gap between the analyser's model of Python semantics and the actual runtime behaviour exposed by BugsInPy's real-world bugs.

4.4 Example: Ansible Bug #11

To illustrate how the self-improvement loop works in practice, consider Ansible bug #11 from BugsInPy. The buggy code fails to check whether a pattern string contains certain special characters before passing it to a regex-based host-matching function, causing an unhandled `re.error` exception. The fix adds an explicit guard:

```
# Fixed version (simplified)
def _match_one_pattern(pattern, host):
    if '[' in pattern:
        pattern = _expand_brackets(pattern)
    # ... regex matching ...
```

A3 initially missed this bug (false negative) because its exception-type detectors did not model `re.error` as a distinct exception class

reachable from `re.compile()` and `re.match()` when the input pattern is user-controlled. The self-improvement protocol constructed a prompt describing the gap—"`re.compile()` can raise `re.error` when given a malformed pattern; A3 does not currently model this"—and the LLM coding assistant added a contract for `re.compile` to the library-contracts module. After the patch, A3 correctly flags the buggy version and passes the fixed version.

4.5 Cumulative Effect

Over hundreds of iterations, the self-improvement loop has driven three categories of analyser improvements:

(1) **Library contracts** (42% of patches): adding or refining pre/post-conditions for standard-library and third-party functions (e.g., `re.compile`, `json.loads`, `os.path.join`, `dict.pop`).
(2) **Guard-detection patterns** (31% of patches): teaching the guard detector to recognise new idioms (e.g., `getattr(x, attr, default)`, `x = x or fallback`, walrus-operator guards `if (m := re.match(...)) is not None`).
(3) **Bug-type refinements** (27% of patches): splitting coarse bug types into finer categories (e.g., `PANIC` → `VALUE_ERROR` + `RUNTIME_ERROR` + `FILE_NOT_FOUND`) and mapping each to appropriate verification strategies.

This cumulative patching is what distinguishes the inverted loop from one-shot analyser development: each BugsInPy bug that the analyser gets wrong becomes a permanent regression test, and the fix is automatically verified before the loop advances.

5 Continuous Verification via Git History

The BugsInPy self-improvement loop (§4) relies on a curated benchmark with labelled buggy/fixed commit pairs. A natural question is whether the same methodology can be applied to *arbitrary* Python repositories where no such labels exist. A3 answers this affirmatively by mining bug-fix commits directly from a repository's git history.

5.1 Commit-Level Bug Detection

Given any Python repository, A3 identifies candidate bug-fix commits through a four-phase pipeline:

(1) **Clone and enumerate.** The repository is cloned (with configurable `--depth` for efficiency) and its commit history is traversed.
(2) **Classify commits.** Each commit's message and diff are analysed using keyword heuristics and AST-level patch inspection to determine whether the commit is likely a bug fix. Signals include: commit messages containing "fix", "bug", "crash", "error", or exception-type keywords; patches that add guard checks (`if x is None`, `isinstance`, bounds checks); patches that modify exception-handling code (`try/except/raise`); and patches that touch security-relevant APIs. A confidence score is assigned to each classification.
(3) **Diff-level scanning.** For each candidate bug-fix commit, A3 extracts the changed Python files at both the parent (buggy) and child (fixed) revisions, and runs the full analysis pipeline on both versions.
(4) **Ground-truth comparison.** Results are classified against the commit's implicit ground truth: a true positive means A3 reports a bug in the parent version and reports clean in the fixed version; a false negative means the parent version scans clean; a false positive means the fixed version is still flagged.

This pipeline requires no manual annotation: the git history *itself* serves as a continuously growing ground-truth corpus.

5.2 From Commits to Analyser Improvements

Misclassifications discovered by commit-level scanning feed directly into the self-improvement loop described in §4. For each false negative or false positive, a structured prompt is generated describing the gap—the expected bug type, the relevant code pattern, and the commit's patch—and an LLM coding assistant proposes a patch to A3's implementation. The patch is verified by re-scanning the commit pair before acceptance.

This creates a *continuous verification* cycle: as A3 is deployed across repositories (whether via CI integration or batch evaluation), every bug-fix commit it encounters becomes a potential improvement signal. The analyser does not merely scan code—it *learns from the corrections that developers make*, using each repository's maintenance history as training data for its own refinement.

5.3 Scalability and Convergence

To avoid unbounded iteration, the pipeline monitors improvement convergence via plateau detection: when successive iterations over a repository's commit history yield diminishing returns (fewer new misclassifications resolved per iteration), the loop terminates. State is checkpointed to JSON between iterations, enabling resumption across sessions and incremental re-analysis as new commits arrive.

In practice, repositories with rich exception-handling and guard patterns (e.g., Ansible, Tornado, Scrapy) yield the most improvement signal, while repositories whose bugs are primarily algorithmic (e.g., Black, tqdm) contribute fewer but still non-zero improvements—typically to library-contract coverage and guard-detection heuristics.

5.4 Example: Keras Bug-Fix Mining

To illustrate the pipeline on a repository outside BugsInPy's curated set, consider applying commit-level scanning to the Keras repository. A commit with message "fix: handle None layer input in __call__" modifies the `__call__` method to add an explicit `if inputs is None: raise ValueError(...)`. The pipeline extracts the parent commit (buggy, missing the guard) and the child commit (fixed, with the guard), then scans both:

- On the parent commit, A3 reports `NULL_PTR` on the attribute access `inputs.shape`—a true positive.
- On the child commit, the finding disappears because the guard check prevents the null flow.

This is a correct classification, so no analyser improvement is triggered. However, a different commit—one that adds a `hasattr` guard instead of an `isinstance` check—initially produces a false positive on the fixed version because A3's guard detector does not recognise `hasattr(x, 'shape')` as a null guard for subsequent

Table 1: Synthetic benchmark results.

Suite	TP	FP	FN	TN	Prec.	Recall
Standalone	29	2	0	2	93.5%	100%
Multi-file	8	2	0	0	80.0%	100%
Combined	**37**	**4**	**0**	**2**	**90.2%**	**100%**

`x.shape` access. The self-improvement loop generates a prompt describing this gap, the LLM proposes a patch to the `hasattr`-based guard-detection heuristic, and after verification, the patch is accepted. This improvement then applies globally: all future scans across all repositories benefit from the new `hasattr` guard pattern.

6 Evaluation

6.1 Synthetic Benchmark

We constructed a synthetic benchmark of 43 programs spanning 67 bug types, including both standalone single-file programs and multi-file projects with interprocedural bug flows. Each program has a known ground-truth label (BUG or SAFE) and a documented bug type.

As shown in Table 1, A3 achieves 90.2% precision at perfect recall, yielding an F_1 score of 0.949. The 4 false positives stem from conservative handling of opaque library calls; the analyser reports a potential bug rather than assuming safety for unknown code.

The benchmark covers the following bug families:

- **Injection**: SQL injection via string concatenation (3 variants), command injection via `os.system` and `subprocess` (3 variants), code injection via `eval` (2 variants), including interprocedural flows up to 4 hops.
- **Crash**: division-by-zero (8 variants including floor division, modulo, and average-of-empty-list), null-pointer / `AttributeError` (6 variants), out-of-bounds (5 variants), assertion failures (2 variants).
- **Serialisation**: `pickle.load` of untrusted data, `yaml.load` without `SafeLoader`, XXE via `xml.etree`.
- **Safe programs**: 2 programs using parameterised queries and input sanitisation, correctly classified as safe (true negatives).

The standalone suite tests single-function and single-file detection; the multi-file suite tests interprocedural taint tracking across module boundaries (e.g., taint flows from `flask.request.args` in `app.py` through a helper in `utils.py` to a SQL query in `db.py`).

6.2 DeepSpeed Case Study

We ran A3 on Microsoft DeepSpeed, a widely-used deep learning optimisation library with approximately 150 k lines of Python across 7 826 functions.

Of the 4 261 raw candidates, 95.7% (4 077) were proven false positives by guard detection and barrier certificates. Of the surviving 184 candidates subjected to deep manual investigation, we confirmed:

- **6 genuine bugs**: unguarded division-by-zero reachable from user-supplied configuration values.

Table 2: DeepSpeed false-positive elimination pipeline.

Stage	Findings remaining
Raw candidates	4 261
After guard detection (Papers #1–20)	342
After barrier certificates (B_1–B_8)	125
After DSE (Z3)	∼10

- **4 borderline bugs**: division-by-zero reachable only from malformed configuration.
- **174 false positives**: attributed to framework guarantees (`self` never `None`), pytest fixtures, argparse defaults, and HuggingFace config invariants.

Example: `_ensure_divisibility`. In `deepspeed/utils/groups.py`, the function:

```
def _ensure_divisibility(numerator,
    denominator):
    assert numerator % denominator == 0, \
        '{} is not divisible by {}'.format(
            numerator, denominator)
```

is called 7 times with user-configurable parallel sizes. If a user passes `model_parallel_size=0` in their DeepSpeed JSON configuration, `numerator % denominator` raises `ZeroDivisionError` *before* the assertion message fires.

Example: `ThroughputTimer`. In `deepspeed/utils/timer.py`:

```
def _is_report_boundary(self):
    if self.steps_per_output is None:
        return False
    return self.global_step_count \
           % self.steps_per_output == 0
```

The `None` check passes when `steps_per_output=0`, but the subsequent modulo crashes.

Example: Silent corruption in FP quantisation. In `op_builder/hpu/fp_quantize`

```
dequant_out = torch.ops.hpu.cast_from_fp8(
    input_q, (1.0 / scale), orig_dtype)
```

If any quantisation group has all-zero values, `scale` is zero, producing `inf` or `nan` that *silently* corrupts model weights—no crash, no error message, just wrong inference results.

6.3 Microsoft Repository Portfolio

We evaluated A3 across 15 Microsoft-maintained open-source repositories. Table 3 summarises the confirmed true positives.

Counterfit: `eval()` on CLI input. Azure Counterfit, an AI security testing framework, uses `eval()` to parse user-provided sample indices, clip values, and numeric parameters directly from CLI arguments:

```
def get_sample_index(sample_index: str):
    try:
        sample_index = eval(sample_index)
    except Exception as e:
        ...
```

Table 3: True positives confirmed across Microsoft repositories.

Repository	Bug type	#TPs	Severity
Counterfit	CODE_INJECTION	4	High
RD-Agent	TAR_SLIP	2	High
RD-Agent	CODE_INJECTION	1	Med–High
RESTler	COMMAND_INJECTION	3	Med–High
FLAML	DIV_ZERO	2	Medium
FLAML	BOUNDS	1	Low
Presidio	REGEX_INJECTION	1	Medium
DeepSpeed	UNSAFE_DESER.	1	Medium
DeepSpeed	DIV_ZERO	6	Medium
GraphRAG	PATH_INJECTION	1	Low
Total		**22**	

A user can pass `--sample_index "__import__('os').system('id')"` to achieve arbitrary code execution. Although Counterfit is a CLI tool (limiting remote exploitability), shared automation scripts that invoke it with external input are vulnerable.

RD-Agent: tarfile path traversal. Microsoft's RD-Agent downloads tar archives from Kaggle competitions and extracts them without path validation:

```
with tarfile.open(tar_path, mode="r:*") as tar
    :
    tar.extractall(path=to_dir)  # no filter!
```

A malicious archive containing entries with `../` prefixes can write files outside the target directory—including `.bashrc` or `.ssh/authorized_keys`. Python 3.12 added `filter='data'` to mitigate this, but the code does not use it.

FLAML: division by zero in feature normalisation. FLAML's meta-learning suggest module normalises features by a precomputed scale:

```
feature = (np.array(feature) - center) \
          / np.array(prep["scale"])
```

If any training feature is constant, `prep["scale"]` contains a zero element, causing `ZeroDivisionError` (or silent `nan` propagation with NumPy).

6.4 Comparison with CodeQL

A3 implements a mapping from CodeQL's Python security query suite to its own detectors, covering injection, deserialisation, XSS, SSRF, path traversal, cryptographic, and configuration bug classes. The security detector registry covers all 39 CodeQL-equivalent security bug types plus 28 additional crash, exception, and semantic bug types that CodeQL's Python queries do not address.

The key architectural difference is that CodeQL requires writing queries in a custom language (QL) and running a separate database-extraction step, whereas A3 operates directly on Python bytecode with no intermediate representation. CodeQL's dataflow framework is more mature for large-scale taint tracking; A3's advantage is the barrier-certificate layer, which aggressively eliminates false positives that CodeQL's queries would surface. In the DeepSpeed evaluation, a hypothetical CodeQL scan (simulated by running A3 without barrier certificates) would produce 342 unguarded findings; with barriers, only 125 survive—a 63.5% reduction in noise that CodeQL's framework does not provide.

7 Agentic Triage

Findings that survive the static pipeline—typically around 1% of raw candidates—are forwarded to an *agentic LLM triage* stage. Rather than asking an LLM to classify a finding from a one-shot prompt, A3 equips the LLM with tools to *explore* the codebase:

- `read_file`: read specific source ranges around the flagged location.
- `search_codebase`: grep for guard patterns, callers, imports, and related symbols.
- `get_function_source`: retrieve the full function body, including decorators and docstrings.
- `get_imports`: list all imports in a module to understand dependencies.
- `list_directory`: explore project structure for test files, configuration, and related modules.

The LLM agent conducts a multi-turn investigation—reading the flagged function, checking callers, inspecting tests, following imports—before producing a TP/FP classification with a rationale. This agentic approach replaces the fragile one-shot classification that characterises most LLM-triage systems.

Example triage session. Consider a `NULL_PTR` finding in `deepspeed/runtime/en` on line 1024, where `self.optimizer` is accessed without a prior null check. The agentic triage proceeds as follows:

(1) The agent reads lines 1020–1030 and sees `self.optimizer.step()`.
(2) It searches for `self.optimizer =` across the codebase and finds that $_{init_{alwaysassignsself.optimizertoaconcreteoptimiserobject(neverNone)}}$
(2) It checks whether any method sets `self.optimizer = None` and finds none.
(3) It classifies the finding as **FP** with rationale: "`self.optimizer` is assigned a non-None value in $_{init_{andisneverreassignedtoNone.''}}$

This multi-step reasoning is infeasible in a one-shot prompt because the relevant evidence (the constructor, the absence of null assignments) is scattered across thousands of lines.

Triage accuracy. On the DeepSpeed residual (125 findings after barrier-certificate elimination), agentic triage correctly classified 119 of 125 findings (95.2% accuracy), with 4 false positives incorrectly marked as true positives and 2 true positives incorrectly dismissed. The multi-turn investigation format is critical to this accuracy: in an ablation where the agent was given only the flagged line and function body (simulating one-shot classification), accuracy dropped to 71.2%. The difference is attributable to findings whose resolution requires cross-file evidence—constructor invariants, callee return guarantees, and framework conventions—that a one-shot prompt cannot access.

Provider flexibility. In CI (GitHub Actions), the triage uses the `GITHUB_TOKEN` already available on every runner to access GitHub

Models, requiring zero additional API key configuration. Locally, developers can use OpenAI, Anthropic, or GitHub Models by setting the corresponding environment variable. The triage is provider-agnostic: the same tool-use protocol works across all supported LLMs.

8 CI Integration and Baseline Ratcheting

A practical design choice that makes A3 deployable in large existing codebases is *baseline ratcheting*:

- Existing accepted findings are recorded in `.a3-baseline.json`.
- New unaccepted findings fail CI.
- Disappearing findings are auto-pruned from the baseline.

This shifts the developer experience from "infinite backlog" to "no net new risk," which is the only sustainable adoption model for large existing codebases. Adding A3 to any repository takes 60 seconds:

```
pip install a3-python[ci]
a3 init . --copilot
git add .github/ .a3.yml .a3-baseline.json
git commit -m "ci: add a3 static analysis"
git push
```

Workflow details. The `a3 init --copilot` command generates two GitHub Actions workflows:

(1) **PR scan** (`a3-pr-scan.yml`): triggers on every push to `main`/`master` and every pull request that modifies `.py` files. It runs the full static pipeline, invokes agentic triage, checks the result against the baseline, and uploads SARIF to GitHub's Code Scanning dashboard.
(2) **Scheduled scan** (`a3-scheduled-scan.yml`): runs weekly (Monday 6 AM UTC) on the full repository. New true positives are automatically filed as GitHub Issues with the finding details, affected file, and suggested fix.

SARIF output. A3 outputs findings in SARIF v2.1.0 (Static Analysis Results Interchange Format), the standard consumed by GitHub Code Scanning, Azure DevOps, and VS Code's SARIF Viewer extension. Each SARIF result includes the bug type, confidence score, affected source range, a human-readable message, and—when available—a counterexample trace showing the path from function entry to the bug site. The SARIF file serves as the interchange format between the static pipeline and the agentic triage: triage reads findings from SARIF, annotates each with a TP/FP verdict and rationale, and writes a filtered SARIF file containing only confirmed true positives.

Connecting CI to continuous verification. When deployed in CI, A3 naturally observes the repository's evolving commit history. Each pull request that fixes a bug—whether flagged by A3 or discovered independently—produces a buggy/fixed commit pair that can feed back into the self-improvement loop (§5). The scheduled weekly scan serves a dual purpose: it catches new bugs introduced by dependency updates or configuration drift, and it periodically re-evaluates the repository's commit history for new improvement signals. This design means that CI adoption is not merely a deployment mechanism but an active contributor to the virtuous cycle: the more repositories run A3 in CI, the more bug-fix commits flow through the improvement pipeline, and the stronger the analyser becomes for all users.

9 Related Work

Python Static Analysis. Pyright [?], mypy [?], and Pytype [?] focus on type checking rather than bug detection; they excel at catching type errors in annotated code but do not target crash bugs (division by zero, null dereference) or security vulnerabilities (injection, deserialisation). Bandit [?] applies pattern-matching (AST visitor rules) for security issues but produces high false-positive rates because it lacks semantic analysis—it cannot determine whether a flagged `eval()` call is reachable from user input. CodeQL [?] provides a powerful dataflow framework with low false-positive rates for the queries it ships, but requires writing queries in a custom language (QL), runs a separate database-extraction step, and does not target crash bugs. Semgrep [?] offers lightweight pattern matching with a lower barrier to entry than CodeQL but similarly lacks the formal FP-elimination layer. A3 complements these tools by combining crash, security, and semantic bug detection with barrier-certificate-based false-positive elimination.

Barrier Certificates. Prajna et al. [2, 3] introduced barrier certificates for hybrid system safety verification. Subsequent work extended barriers to stochastic systems [3], polynomial optimisation via SOS/SDP [5], and compositional verification [18]. We adapt the core idea—constructing a separating function between safe and unsafe regions—to the discrete, exception-rich domain of Python program states, where the "state space" is the set of possible variable bindings at each program point and the "unsafe region" is the set of bindings that trigger an unhandled exception.

CEGAR and IC3/PDR.. Counterexample-guided abstraction refinement [12] and property-directed reachability [9] are the workhorses of hardware and C-program model checking. A3's kitchensink engine invokes these techniques as components in a portfolio rather than as standalone tools, routing each finding to the cheapest technique likely to resolve it.

LLM-Assisted Analysis. Li et al. [?] explored using LLMs to generate static-analysis rules and found that fewer than 30% of generated rules were sound—consistent with our observation that the AI-slop feedback loop is the default outcome of unchecked LLM-assisted tooling. Other work has used LLMs to triage findings [?], repair bugs [?], and generate test cases. A3 differs in placing LLM judgment *after* deterministic static analysis, not instead of it, and in using the LLM's coding ability to *improve the analyser itself* rather than only to classify individual findings.

BugsInPy. Widyasari et al. [1] curated BugsInPy as a benchmark for testing and debugging research. Prior work has used BugsInPy primarily for evaluating fault localisation and automated program repair tools. We use it not only for evaluation but as a *self-improvement driver*: classification errors on BugsInPy bugs trigger automated patches to the analyser, making the benchmark an active participant in the development loop rather than a passive test suite.

Invariant Synthesis. Houdini [16] pioneered conjunctive invariant inference; ICE [15] extended this to a teacher-learner framework with implication counterexamples; SyGuS [17] generalised synthesis to arbitrary grammars. A3 treats these as interchangeable invariant-proposal engines within the kitchensink portfolio, selecting among them based on the bug type and the complexity of the surrounding control flow.

10 Threats to Validity

Internal. The synthetic benchmark was constructed by the authors and may not represent the full diversity of real-world bugs. We mitigate this with the BugsInPy and DeepSpeed evaluations, which use independently curated bug databases and real industrial code.

External. Results on DeepSpeed and 15 Microsoft repositories may not generalise to all Python codebases, particularly those with heavy metaprogramming, C extensions, or dynamic code generation. Python's dynamic nature means that any static analyser must make conservative assumptions about runtime behaviour; A3's contract system mitigates this but cannot eliminate it.

Construct. The distinction between "true positive" and "false positive" for division-by-zero bugs depends on whether user-configurable values can realistically be zero. We report borderline cases separately. For security bugs, exploitability assessments are subjective; we report estimated likelihood but acknowledge that operational context (e.g., whether a CLI tool is exposed to network input) affects real-world severity.

11 Discussion

The role of AI in the loop. A natural question is whether the AI components of A3 are essential or merely convenient. We argue they are essential at *two* points and dispensable everywhere else. First, during development, LLM-generated hypotheses dramatically expand the space of candidate analysis techniques explored per unit time; a human researcher would take months to propose and prototype the 18 techniques in the kitchensink portfolio, whereas the LLM proposes candidates in minutes (most of which are wrong, but the survivors are valuable). Second, during triage, the agentic LLM resolves the long tail of findings that resist automated proof or disproof—typically 1–3% of raw candidates—by reading code, following imports, and reasoning about domain-specific invariants (e.g., "HuggingFace configs always set `num_attention_heads` ≥ 1").

At every other point in the pipeline, AI is deliberately excluded: CFG construction, guard detection, barrier-certificate proofs, DSE confirmation, and deduplication are all deterministic and auditable. This separation is what prevents the virtuous cycle from reverting to a vicious one.

Scalability. A3's analysis time is dominated by the Z3-backed DSE stage. On DeepSpeed (7 826 functions), a full scan completes in approximately 8 minutes on a single core. The barrier-certificate stage adds less than 10% overhead because most barriers are resolved by lightweight intraprocedural checks (B_6, B_7) before expensive interprocedural checks (B_1, B_8) are attempted. Agentic triage is the slowest stage (1–5 seconds per finding due to LLM latency), but because it processes only the 1% residual, wall-clock time remains acceptable for CI use.

Continuous verification in practice. The generalisation from BugsInPy to arbitrary git histories (§5) transforms A3 from a static tool into one that improves with deployment. Every repository scanned in CI contributes potential improvement signals: bug-fix commits expose patterns that the analyser misclassifies, and those misclassifications drive targeted patches. This creates a flywheel effect—the more repositories adopt A3, the more bug-fix commits it observes, and the stronger its detection becomes for the next repository. The practical implication is that organisations running A3 across a portfolio of repositories benefit not only from per-repo scanning but from cross-repo learning, as guard patterns and library contracts discovered in one project transfer to all others.

Limitations of the self-improvement loop. The BugsInPy-driven self-improvement loop is not a silver bullet. It can only improve A3 for bug patterns that appear in BugsInPy's 17 projects; however, the continuous verification extension (§5) substantially mitigates this limitation by mining bug-fix commits from arbitrary repositories, expanding the effective training corpus beyond any single curated benchmark. Entirely novel vulnerability classes (e.g., prompt injection in LLM applications) still require manual extension. Additionally, the loop's reliance on an LLM coding assistant means that patches are not guaranteed to be correct—each patch must pass the verification step before being accepted, and approximately 30% of LLM-proposed patches fail verification and are discarded.

12 Conclusion

The AI-slop feedback loop—in its original, unchecked form—is a vicious cycle that degrades every tool it touches. But it is not inevitable. By placing deterministic static analysis—barrier certificates, Z3-backed symbolic execution, and a portfolio of 18 verification techniques—*before* LLM judgment, A3 inverts the loop: each iteration destroys noise rather than propagating it, and the surviving signal feeds back into better hypotheses, turning a historically negative phenomenon into a virtuous self-improvement cycle.

On synthetic benchmarks, A3 achieves $F_1 = 0.949$. On DeepSpeed, the barrier-certificate engine alone eliminates 95.3% of false positives, leaving confirmed bugs that include division-by-zero vulnerabilities reachable from user configuration and a silent data-corruption issue in FP quantisation. The BugsInPy self-improvement protocol demonstrates that real-world bug databases can drive automated analyser patches, closing the gap between the analyser's model and Python's runtime semantics. By generalising this protocol to mine bug-fix commits from arbitrary Python repositories, A3 enables continuous verification: the analyser improves with every repository it scans, turning each project's git history into training signal for stronger detection across all future analyses.

A3 is available at `pip install a3-python` under an open-source licence.

References

[1] R. Widyasari, S. Q. Sim, C. Lok, H. Qi, J. Phan, Q. Tran, H. Hata, S. Chandrasekaran, C. Yang, and D. Lo. BugsInPy: A database of existing bugs in Python programs to enable controlled testing and debugging studies. In *Proc. ESEC/FSE*, 2020.
[2] S. Prajna, A. Jadbabaie, and G. J. Pappas. Safety verification of hybrid systems using barrier certificates. In *Proc. HSCC*, 2004.

[3] S. Prajna, A. Jadbabaie, and G. J. Pappas. A framework for worst-case and stochastic safety verification using barrier certificates. *IEEE Trans. Automatic Control*, 52(8):1415–1428, 2007.
[4] M. Putinar. Positive polynomials on compact semi-algebraic sets. *Indiana Univ. Math. J.*, 42(3):969–984, 1993.
[5] P. A. Parrilo. Semidefinite programming relaxations for semialgebraic problems. *Math. Programming*, 96(2):293–320, 2003.
[6] J.-B. Lasserre. Global optimization with polynomials and the problem of moments. *SIAM J. Optim.*, 11(3):796–817, 2001.
[7] M. Kojima, S. Kim, and H. Waki. Sparsity in sums of squares of polynomials. *Math. Programming*, 103(1):45–62, 2005.
[8] A. A. Ahmadi and A. Majumdar. DSOS and SDSOS optimization: more tractable alternatives to sum of squares and semidefinite optimization. *SIAM J. Appl. Algebra Geometry*, 3(2):193–230, 2019.
[9] A. R. Bradley. SAT-based model checking without unrolling. In *Proc. VMCAI*, 2011.
[10] A. Komuravelli, A. Gurfinkel, and S. Chaki. SMT-based model checking for recursive programs. In *Proc. CAV*, 2014.
[11] K. L. McMillan. Interpolation and SAT-based model checking. In *Proc. CAV*, 2003.
[12] E. Clarke, O. Grumberg, S. Jha, Y. Lu, and H. Veith. Counterexample-guided abstraction refinement. In *Proc. CAV*, 2000.
[13] E. Clarke, D. Kroening, N. Sharygina, and K. Yorav. Predicate abstraction of ANSI-C programs using SAT. *Formal Methods in System Design*, 25(2–3):105–127, 2004.
[14] K. L. McMillan. Lazy abstraction with interpolants. In *Proc. CAV*, 2006.
[15] P. Garg, C. Löding, P. Madhusudan, and D. Neider. ICE: A robust framework for learning invariants. In *Proc. CAV*, 2014.
[16] C. Flanagan and K. R. M. Leino. Houdini, an annotation assistant for ESC/Java. In *Proc. FME*, 2001.
[17] R. Alur, R. Bodik, G. Juniwal, M. M. K. Martin, M. Raghothaman, S. A. Seshia, R. Singh, A. Solar-Lezama, E. Torlak, and A. Udupa. Syntax-guided synthesis. In *Proc. FMCAD*, 2013.
[18] T. A. Henzinger, S. Qadeer, and S. K. Rajamani. You assume, we guarantee: methodology and case studies. In *Proc. CAV*, 1998.